

มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน คณะศิลปศาสตร์และวิทยาศาสตร์
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีดิจิทัล

บทปฏิบัติการประกอบเอกสารอ่านนอกเวลา

สร้างแอป Todo ที่จำข้อมูลได้ ด้วย AsyncStorage

สร้างที่ละขั้นจากโครงการเปล่า จนได้แอปที่ใช้งานได้จริง

วิชา 01418342 การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ · Expo SDK 57 · โครงการตัวอย่าง
TodoStorageDemo

สิ่งที่จะได้เมื่อทำครบทุกขั้น

แอปรายการสิ่งที่ต้องทำที่ทำงานได้จริงบนโทรศัพท์ ประกอบด้วย

- เพิ่ม ดีกว่าเสร็จ และลบรายการได้
- ข้อมูลยังอยู่ครบเมื่อปิดแอปแล้วเปิดใหม่
- กรองตามสถานะได้ และแอปจำตัวกรองที่เลือกไว้ล่าสุด
- ลบเฉพาะงานที่เสร็จแล้ว หรือล้างข้อมูลทั้งหมดอย่างปลอดภัย
- ปุ่มสำหรับดูว่ามีอะไรถูกเก็บไว้ในเครื่องบ้าง ใช้ตอนหาสาเหตุบั๊ก

เวลาที่ใช้: ประมาณ 2 ชั่วโมง · เอกสารอ้างอิง: เอกสารอ่านนอกเวลา “การเก็บข้อมูลในเครื่องบน React Native + Expo”

สารบัญ

1	ก่อนเริ่ม	2
2	ขั้นที่ 0 – สร้างโครงการ	2
3	ขั้นที่ 1 – ค่าคงที่ของโครงการ	3
4	ขั้นที่ 2 – ตัวห่อ AsyncStorage	4
5	ขั้นที่ 3 – แอปที่ยังไม่ทำอะไรเลย	5
6	ขั้นที่ 4 – ทำให้ข้อมูลอยู่ถาวร	8
7	ขั้นที่ 5 – แยกคอมโพเนนต์	10
8	ขั้นที่ 6 – ตัวกรองที่แอปจำได้	13
9	ขั้นที่ 7 – ล้างข้อมูลอย่างปลอดภัย	16
10	รายการทดสอบก่อนส่งงาน	20
11	โจทย์ต่อยอด	20
12	ภาคผนวก – โครงการตัวอย่าง	21

1 ก่อนเริ่ม

1.1 สิ่งที่ต้องมี

- Node.js เวอร์ชัน 22.13 ขึ้นไป (ข้อกำหนดขั้นต่ำของ Expo SDK 57)
- แอป Expo Go บนโทรศัพท์ หรือโปรแกรมจำลอง Android/iOS
- โปรแกรมแก้ไขโค้ด แนะนำ Visual Studio Code
- เครื่องคอมพิวเตอร์และโทรศัพท์ต่อ Wi-Fi วงเดียวกัน

1.2 วิธีใช้เอกสารนี้

เอกสารนี้ให้ พิมพ์โค้ดตามที่ระบุ ไม่ใช่คัดลอกวางทั้งก้อน เพราะเป้าหมายคือให้เห็นว่าแอปค่อย ๆ ทำงานได้มากขึ้นอย่างไร ในแต่ละขั้นจะมีกล่อง **ผลที่ควรเห็น** บอกว่าเมื่อทำถูกต้องแล้วหน้าจควรเป็นอย่างไร ถ้าผลไม่ตรง ให้ย้อนกลับไปตรวจขั้นนั้นก่อนไปต่อ

โครงการตัวอย่างที่ทำเสร็จแล้วอยู่ในไฟล์ TodoStorageDemo.zip ให้เปิดดูเมื่อทำแล้วติดขัดจริง ๆ เท่านั้น

2 ขั้นที่ 0 – สร้างโครงการ

เปิดเทอร์มินัลไปยังโฟลเดอร์ที่ต้องการเก็บงาน แล้วพิมพ์

```
npx create-expo-app@latest TodoStorageDemo --template blank
cd TodoStorageDemo
npx expo install @react-native-async-storage/async-storage
```

จากนั้นเริ่มเซิร์ฟเวอร์พัฒนา

```
npx expo start
```

สแกน QR code ด้วยแอป Expo Go จะเห็นหน้าจอเปล่าพร้อมข้อความตั้งต้นของ Expo

ต้องใช้ npx expo install เท่านั้น

อย่าติดตั้ง AsyncStorage ด้วย npm install ตรง ๆ เพราะจะได้เวอร์ชันล่าสุดซึ่งอาจไม่ตรงกับ SDK 57 อาการที่พบคือแอปค้างที่หน้าขาว หรือขึ้นข้อความว่าโมดูลเนทีฟไม่ตรงเวอร์ชัน คำสั่ง npx expo install จะเลือกเวอร์ชันที่ทดสอบแล้วว่าเข้ากันได้ให้เอง

2.1 โครงสร้างไฟล์ที่จะสร้างขึ้น

ตลอดบทปฏิบัติการนี้เราจะสร้างไฟล์ตามโครงนี้ ให้สร้างโฟลเดอร์เปล่าไว้ก่อนได้เลย

```

TodoStorageDemo/
  App.js
  constants/
    colors.js      ชุดสีของแอป
    layout.js      ระยะขอบปลอดภัย
    keys.js        ชื่อคีย์ของ AsyncStorage
  utils/
    storage.js     ตัวห่อ AsyncStorage
  hooks/
    usePersistedState.js
  components/
    TodoInput.js
    TodoItem.js
    FilterBar.js
  screens/
    TodoScreen.js  หน้าจอหลัก รวมตรรกะทั้งหมด

```

การแยกไฟล์แบบนี้ไม่ได้ทำเพื่อความสวยงามอย่างเดียว แต่เพื่อให้ตรรกะการเก็บข้อมูลอยู่รวมกันที่ `utils/` และหน้าจอไม่ต้องรู้ว่าข้อมูลถูกเก็บด้วยกลไกใด

3 ขั้นที่ 1 – ค่าคงที่ของโครงการ

3.1 ชุดสี

สร้างไฟล์ `constants/colors.js` ใช้ชุดสีเดียวกับสไลด์ของวิชา

```

export const COLORS = {
  bg:      '#0D1117',    // พื้นหลังหลัก
  card:    '#161B22',    // พื้นหลังการ์ด
  border:  '#30363D',    // เส้นขอบ
  text:    '#E6EDF3',    // ตัวอักษรหลัก
  textDim: '#8B949E',    // ตัวอักษรรอง
  cyan:    '#61DAFB',    // สีเน้นหลัก
  amber:    '#F0A93B',    // สถานะรอดำเนินการ
  green:    '#3FB950',    // สถานะเสร็จแล้ว
  violet:   '#A371F7',    // ปุ่มรอง
  red:     '#F85149',    // การลบ
};

```

3.2 ระยะขอบปลอดภัย

สร้างไฟล์ `constants/layout.js` เราคำนวณระยะขอบด้านบนเอง โดยไม่ใช้ `react-native-safe-area-context` เพื่อให้เห็นว่าเบื้องหลังไลบรารีนั้นทำอะไร

```
import { Platform, StatusBar } from 'react-native';

export const TOP_INSET = Platform.select({
  ios: 56, // เพื่อ notch
  android: (StatusBar.currentHeight ?? 24) + 10,
  default: 24, // สำหรับเว็บ
});

export const BOTTOM_INSET = Platform.select({
  ios: 28,
  android: 14,
  default: 14,
});
```

3.3 ชื่อคีย์

สร้างไฟล์ `constants/keys.js` นี่คือนิยามที่ค่าที่เรานำมาใช้ เพราะการพิมพ์ชื่อคีย์เป็นสตริงกระจายทั่วโครงการ เป็นบ่อเกิดของบั๊กที่หายากมาก พิมพ์ผิดหนึ่งตัวก็อ่านข้อมูลไม่เจอ โดยไม่มี error ให้เห็นเลย

```
// ทุกคีย์ขึ้นต้นด้วย 'todo:' เพื่อให้ล้างเป็นกลุ่มได้
// โดยไม่แตะข้อมูลที่ไลบรารีอื่นเก็บไว้
export const PREFIX = 'todo:';

export const KEYS = {
  ITEMS: `${PREFIX}items`,
  FILTER: `${PREFIX}filter`,
};
```

4 ขั้นที่ 2 – ตัวห่อ AsyncStorage

สร้างไฟล์ `utils/storage.js` ไฟล์นี้เป็นด่านเดียวที่แอปทั้งแอปจะติดต่อกับ AsyncStorage ทุกที่ที่เหลือจะเรียกผ่านตัวนี้เท่านั้น

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import { PREFIX } from '../constants/keys';

export const storage = {
  async set(key, value) {
    try {
      await AsyncStorage.setItem(key, JSON.stringify(value));
      return true;
    } catch (e) {
      console.warn('[storage.set] ล้มเหลวที่คีย์', key, e);
    }
  }
};
```

```

        return false;
    }
},

async get(key, fallback = null) {
    try {
        const raw = await AsyncStorage.getItem(key);
        // getItem คืน null เมื่อยังไม่เคยเก็บคีย์นี้
        return raw != null ? JSON.parse(raw) : fallback;
    } catch (e) {
        // ข้อมูลเสียหรือรูปแบบไม่ตรง คืนค่าตั้งต้นดีกว่าปล่อยให้แอปพัง
        console.warn('[storage.get] ล้มเหลวที่คีย์', key, e);
        return fallback;
    }
},

async remove(key) {
    try {
        await AsyncStorage.removeItem(key);
        return true;
    } catch (e) {
        console.warn('[storage.remove] ล้มเหลวที่คีย์', key, e);
        return false;
    }
},
};

```

ประโยชน์ที่จะเห็นชัดในภายหลัง

ถ้าวันหนึ่งข้อมูลโตขึ้นจนต้องเปลี่ยนไปใช้ SQLite หรือ MMKV เราแก้แค่ไฟล์นี้ไฟล์เดียว หน้าจอทั้งแอปไม่ต้องแตะเลยแม้แต่บรรทัดเดียว นี่คือเหตุผลที่ควรห่อไลบรารีภายนอกไว้เสมอ

5 ขั้นที่ 3 – แอปที่ยังไม่ทำอะไรเลย

ขั้นนี้เราจะทำให้แอปใช้งานได้ก่อน โดย ยังไม่แตะ AsyncStorage เลย เพื่อให้เห็นชัดว่าปัญหาคืออะไร

5.1 หน้าจอหลักเวอร์ชันแรก

สร้างไฟล์ screens/ToDoScreen.js

```

import { useState } from 'react';
import {
    View, Text, TextInput, Pressable, FlatList, StyleSheet,
} from 'react-native';

```

```

import { COLORS } from '../constants/colors';
import { TOP_INSET } from '../constants/layout';

export default function TodoScreen() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');

  const addTodo = () => {
    const trimmed = text.trim();
    if (trimmed.length === 0) return;
    setTodos((prev) => [
      { id: Date.now().toString(), text: trimmed, done: false },
      ...prev,
    ]);
    setText('');
  };

  const toggleTodo = (id) =>
    setTodos((prev) =>
      prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));

  const deleteTodo = (id) =>
    setTodos((prev) => prev.filter((t) => t.id !== id));

  return (
    <View style={styles.container}>
      <Text style={styles.title}>สิ่งที่ต้องทำ</Text>

      <View style={styles.row}>
        <TextInput
          style={styles.input}
          value={text}
          onChangeText={setText}
          onSubmitEditing={addTodo}
          placeholder="พิมพ์สิ่งที่ต้องทำ..."
          placeholderTextColor={COLORS.textDim}
        />
        <Pressable style={styles.addButton} onPress={addTodo}>
          <Text style={styles.addText}>เพิ่ม</Text>
        </Pressable>
      </View>

      <FlatList
        data={todos}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable
            onPress={() => toggleTodo(item.id)}
            onLongPress={() => deleteTodo(item.id)}
          />

```

```

        style={styles.item}
      >
        <Text style={ [styles.itemText, item.done && styles.itemDone]} >
          {item.text}
        </Text>
      </Pressable>
    )}
  />
</View>
);
}

```

ส่วนของ StyleSheet ต่อท้ายไฟล์เดียวกัน

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: COLORS.bg,
    paddingTop: TOP_INSET, paddingHorizontal: 20 },
  title: { color: COLORS.text, fontSize: 28, fontWeight: '800',
    marginBottom: 16 },
  row: { flexDirection: 'row', gap: 10, marginBottom: 14 },
  input: { flex: 1, backgroundColor: COLORS.card, borderWidth: 1,
    borderColor: COLORS.border, borderRadius: 10,
    paddingHorizontal: 14, paddingVertical: 12,
    color: COLORS.text, fontSize: 16 },
  addButton: { backgroundColor: COLORS.cyan, borderRadius: 10,
    paddingHorizontal: 20, justifyContent: 'center' },
  addText: { color: COLORS.bg, fontSize: 16, fontWeight: '700' },
  item: { backgroundColor: COLORS.card, borderWidth: 1,
    borderColor: COLORS.border, borderRadius: 10,
    padding: 14, marginBottom: 8 },
  itemText: { color: COLORS.text, fontSize: 16 },
  itemDone: { color: COLORS.textDim, textDecorationLine: 'line-through' },
});

```

5.2 เชื่อมเข้ากับ App.js

แก้ไขไฟล์ App.js ให้เหลือเพียงเท่านั้น

```

import { StatusBar } from 'react-native';
import TodoScreen from './screens/TodoScreen';
import { COLORS } from './constants/colors';

export default function App() {
  return (
    <>
      <StatusBar barStyle="light-content" backgroundColor={COLORS.bg} />
      <TodoScreen />
    </>
  );
}

```

```

    </>
  );
}

```

ผลที่ควรเห็น

เพิ่มรายการได้ แต่หนึ่งครั้งเพื่อปิดฆ่า แต่ค้างเพื่อลบ ทุกอย่างทำงานปกติ แต่ ให้ออกปิดแอปจนสุด (ปิดออกจากรายการแอปที่เปิดอยู่) แล้วเปิดใหม่ จะพบว่า รายการทั้งหมดหายไป เพราะ `useState` เก็บค่าไว้ในหน่วยความจำเท่านั้น นี่คือปัญหาที่ขึ้นต่อไปจะแก้

อย่าใช้ `todos.push`

สังเกตว่าทุกฟังก์ชันสร้างอาร์เรย์ก่อนใหม่เสมอ ไม่มีที่ใดแก้ไขอาร์เรย์เดิมโดยตรง เพราะ React ตรวจสอบการเปลี่ยนแปลงด้วยการเปรียบเทียบการอ้างอิงของอ็อบเจกต์ ถ้าเขียน `todos.push(...)` แล้ว `setTodos(todos)` หน้าจอจะไม่อัปเดต และในขั้นถัดไป effect ที่ทำหน้าที่บันทึกข้อมูลก็จะไม่ทำงานตามไปด้วย

6 ขั้นที่ 4 – ทำให้ข้อมูลอยู่ถาวร

หัวใจของบทปฏิบัติการอยู่ที่ขั้นนี้ เราต้องประสานสองสิ่งที่ทำงานคนละจังหวะกัน คือ `useState` ที่อยู่ในหน่วยความจำและเปลี่ยนทันที กับ `AsyncStorage` ที่อยู่บนดิสก์และเป็นอะซิงโครนัส วิธีมาตรฐานคือใช้ `useEffect` สองก้อน

แก้ไขไฟล์ `screens/ToDoScreen.js` เพิ่ม import และ state ใหม่

```

import { useState, useEffect } from 'react';
import { ActivityIndicator } from 'react-native'; // เพิ่มในรายการ import เดิม
import { storage } from '../utils/storage';
import { KEYS } from '../constants/keys';

```

จากนั้นเพิ่มโค้ดต่อไปนี้ต่อบรรทัด `const [text, setText] = useState('');`

```

const [loading, setLoading] = useState(true); // สำคัญมาก อธิบายด้านล่าง

// (1) โหลดข้อมูลเดิมครั้งเดียวตอนเปิดแอป
useEffect(() => {
  let cancelled = false;

  (async () => {
    const saved = await storage.get(KEYS.ITEMS, []);
    if (!cancelled) {
      setTodos(saved);
      setLoading(false);
    }
  })();
}, []);

```



```

    }
  })();

  return () => { cancelled = true; }; // กัน setState หลัง unmount
}, []);

// (2) บันทึกทุกครั้งที่รายการเปลี่ยน แต่ต้องรอให้โหลดเสร็จก่อน
useEffect(() => {
  if (!loading) storage.set(KEYS.ITEMS, todos);
}, [todos, loading]);

```

และเพิ่มการแสดงผลระหว่างรอโหลด ก่อนคำสั่ง return หลักของคอมโพเนนต์

```

if (loading) {
  return (
    <View style={[styles.container, styles.center]}>
      <ActivityIndicator size="large" color={COLORS.cyan} />
      <Text style={styles.loadingText}>กำลังอ่านข้อมูลจากเครื่อง...</Text>
    </View>
  );
}

```

พร้อมเพิ่มสองสไตล์นี้เข้าไปใน StyleSheet ของไฟล์เดียวกัน

```

center: { alignItems: 'center', justifyContent: 'center', gap: 12 },
loadingText: { color: COLORS.textDim, fontSize: 14 },

```

ผลที่ควรเห็น

เพิ่มรายการสัก 3 รายการ ตีกว่าเสร็จไปหนึ่งรายการ แล้วปิดแอปจนสุดและเปิดใหม่ คราวนี้ทุกอย่างต้องยังอยู่ครบเหมือนเดิม รวมถึงสถานะดีกด้วย
ถ้ายังหาย ให้ตรวจว่าเรียก storage.set ด้วยคีย์เดียวกับที่ storage.get ใช้อ่านหรือไม่

6.1 การทดลองที่ต้องลองด้วยตัวเอง

ทดลองกับดัก hydration

ลองลบเงื่อนไข if (!loading) ใน effect ก่อนที่สอง ให้เหลือเพียง

```

useEffect(() => {
  storage.set(KEYS.ITEMS, todos);
}, [todos]);

```

แล้วปิดแอปเปิดใหม่ จะพบว่าข้อมูลหายทุกครั้ง ทั้งที่ได้ดูถูกต้องทุกประการและ ไม่มี error แม้แต่บรรทัดเดียว

ลำดับเหตุการณ์ที่เกิดขึ้นจริงคือ

1. render รอบแรก todos มีค่าเป็นอาร์เรย์ว่างตามค่าเริ่มต้นของ useState
2. effect ก่อนที่สองทำงานทันที เขียนอาร์เรย์ว่างทับข้อมูลเดิมในเครื่อง
3. effect ก่อนแรกเพิ่งอ่านค่าเสร็จ แต่สิ่งที่อ่านได้คืออาร์เรย์ว่างที่เพิ่งถูกเขียนทับไป

เมื่อเห็นอาการแล้ว ให้ใส่เงื่อนไขกลับคืน บั๊กชนิดนี้เป็นบั๊กที่นิสิตติดกันมากที่สุดในงานชิ้นนี้ และหาสาเหตุยากมากถ้าไม่เคยเห็นมาก่อน

7 ขั้นที่ 5 – แยกคอมโพเนนต์

ตอนนี้ TodoScreen.js เริ่มยาวและปนกันระหว่างตรรกะกับหน้าตา ให้แยกส่วนที่ใช้ซ้ำออกไป

7.1 ช่องกรอกงานใหม่

สร้างไฟล์ components/TodoInput.js คอมโพเนนต์นี้ดูแล state ของข้อความที่กำลังพิมพ์เอง แล้วส่งข้อความที่พร้อมใช้กลับออกมาผ่าน onAdd

```
import { useState } from 'react';
import { View, TextInput, Pressable, Text, StyleSheet } from 'react-native';
import { COLORS } from '../constants/colors';

export default function TodoInput({ onAdd }) {
  const [text, setText] = useState('');

  const handleAdd = () => {
    const trimmed = text.trim();
    if (trimmed.length === 0) return;
    onAdd(trimmed);
    setText('');
  };

  const disabled = text.trim().length === 0;

  return (
    <View style={styles.row}>
      <TextInput
        style={styles.input}
        value={text}
        onChangeText={setText}
        onSubmitEditing={handleAdd}
        placeholder="พิมพ์สิ่งที่ต้องทำ..."
        placeholderTextColor={COLORS.textDim}
        returnKeyType="done"
        maxLength={120}
      />
    </View>
  );
}
```

```

    />
    <Pressable
      onPress={handleAdd}
      disabled={disabled}
      style={({ pressed }) => [
        styles.button,
        disabled && styles.buttonDisabled,
        pressed && !disabled && styles.buttonPressed,
      ]}
    >
      <Text style={styles.buttonText}>เพิ่ม</Text>
    </Pressable>
  </View>
);
}

```

ส่วน styles ให้ย้ายมาจากไฟล์เดิม โดยเปลี่ยนชื่อ addButton เป็น button และเพิ่มสองสไตล์ใหม่

```

buttonDisabled: { backgroundColor: COLORS.border },
buttonPressed: { opacity: 0.75 },

```

ทำไมต้องส่งฟังก์ชันเข้าไปแทนที่จะส่ง state

คอมโพเนนต์ลูกไม่จำเป็นต้องรู้ว่ารายการทั้งหมดมีอะไรบ้าง มันมีหน้าที่เดียวคือรับข้อความแล้วส่งกลับ การออกแบบแบบนี้ทำให้ทดสอบง่ายและนำไปใช้ที่อื่นได้ทันที

7.2 รายการหนึ่งแถว

สร้างไฟล์ components/ToDoItem.js คราวนี้เพิ่มกล่องติ๊กถูกที่วาดเองด้วย View และปุ่มลบแยกต่างหากแทนการแตะค้าง

```

import { View, Text, Pressable, StyleSheet } from 'react-native';
import { COLORS } from '../constants/colors';

export default function ToDoItem({ item, onToggle, onDelete }) {
  return (
    <Pressable
      onPress={() => onToggle(item.id)}
      style={({ pressed }) => [styles.card, pressed && styles.cardPressed]}
    >
      <View style={[styles.checkbox, item.done && styles.checkboxDone]}>
        {item.done && <Text style={styles.checkmark}>{'\u2713'}</Text>}
      </View>

      <Text style={[styles.text, item.done && styles.textDone]}>
        ↵ numberOfLines={2}>

```

```

        {item.text}
      </Text>

      <Pressable
        onPress={() => onDelete(item.id)}
        hitSlop={10}
        style={({ pressed }) => [styles.delete, pressed && { opacity: 0.6 }]}
      >
        <Text style={styles.deleteText}>ลบ</Text>
      </Pressable>
    </Pressable>
  );
}

```

```

const styles = StyleSheet.create({
  card: { flexDirection: 'row', alignItems: 'center', gap: 12,
    backgroundColor: COLORS.card, borderWidth: 1,
    borderColor: COLORS.border, borderRadius: 10,
    paddingHorizontal: 14, paddingVertical: 14, marginBottom: 8 },
  cardPressed: { borderColor: COLORS.cyan },
  checkbox: { width: 22, height: 22, borderRadius: 6, borderWidth: 2,
    borderColor: COLORS.textDim,
    alignItems: 'center', justifyContent: 'center' },
  checkboxDone: { backgroundColor: COLORS.green, borderColor: COLORS.green },
  checkmark: { color: COLORS.bg, fontSize: 14, fontWeight: '900',
    lineHeight: 16 },
  text: { flex: 1, color: COLORS.text, fontSize: 16 },
  textDone: { color: COLORS.textDim, textDecorationLine: 'line-through' },
  delete: { paddingHorizontal: 6, paddingVertical: 2 },
  deleteText: { color: COLORS.red, fontSize: 14, fontWeight: '600' },
});

```

Pressable ซ้อน Pressable

ปุ่มลบเป็น Pressable ที่อยู่ข้างใน Pressable ของทั้งแถว React Native จัดการให้เองว่าการแตะที่ปุ่มลบจะไม่ทำให้แถวถูกตัดไปด้วย แต่พื้นที่แตะของปุ่มเล็กมาก จึงต้องใส่ hitSlop เพื่อขยายพื้นที่รับการแตะออกไปอีก 10 หน่วยทุกด้าน โดยที่หน้าตาไม่เปลี่ยน

7.3 ปรับหน้าจอหลักให้ใช้คอมโพเนนต์ใหม่

ในไฟล์ screens/ToDoScreen.js ให้ลบ TextInput และ state text ออก เพราะย้ายไปอยู่ใน TodoInput แล้ว จากนั้นเปลี่ยนส่วนแสดงผลเป็น

```

<TodoInput onAdd={addTodo} />

<FlatList

```

```

data={todos}
keyExtractor={(item) => item.id}
renderItem={({ item }) => (
  <TodoItem item={item} onToggle={toggleTodo} onDelete={deleteTodo} />
)}
keyboardShouldPersistTaps="handled"
/>

```

และแก้ไข `addTodo` ให้รับข้อความเข้ามาแทนการอ่านจาก state ของตัวเอง

```

const addTodo = (text) => {
  setTodos((prev) => [
    { id: `${Date.now()}-${Math.random().toString(36).slice(2, 7)}`,
      text, done: false, createdAt: Date.now() },
    ...prev,
  ]);
};

```

ทำไม id ต้องมีส่วนสุ่มด้วย

ถ้าใช้ `Date.now()` อย่างเดียว แล้วผู้ใช้กดเพิ่มสองรายการเร็วมากภายในมิลลิวินาทีเดียวกัน จะได้ id ซ้ำกัน ทำให้ `FlatList` เตือนเรื่อง key ซ้ำ และการคลิกหรือลบจะไปโดนผิดรายการ การต่อท้ายด้วยค่าสุ่มสั้น ๆ ตัดปัญหานี้ได้

8 ขั้นที่ 6 – ตัวกรองที่แอปจำได้

ขั้นนี้จะได้ใช้ custom hook ที่ทำให้โค้ดสั้นลงอย่างเห็นได้ชัด

8.1 สร้าง `usePersistedState`

สร้างไฟล์ `hooks/usePersistedState.js` สังเกตว่าเนื้อในคือรูปแบบเดียวกับที่เขียนไว้ในขั้นที่ 4 ทุกประการ เพียงแต่ยกออกมาให้ใช้ซ้ำได้

```

import { useState, useEffect } from 'react';
import { storage } from '../utils/storage';

export function usePersistedState(key, initialValue) {
  const [value, setValue] = useState(initialValue);
  const [hydrated, setHydrated] = useState(false);

  useEffect(() => {
    let cancelled = false;
    (async () => {

```

```

    const saved = await storage.get(key, initialValue);
    if (!cancelled) {
      setValue(saved);
      setHydrated(true);
    }
  })();
  return () => { cancelled = true; };
}, [key]);

useEffect(() => {
  if (hydrated) storage.set(key, value);
}, [key, value, hydrated]);

return [value, setValue, hydrated];
}

```

8.2 แถบตัวกรอง

สร้างไฟล์ components/FilterBar.js

```

import { View, Text, Pressable, StyleSheet } from 'react-native';
import { COLORS } from '../constants/colors';

export const FILTERS = [
  { key: 'all', label: 'ทั้งหมด' },
  { key: 'active', label: 'ยังไม่เสร็จ' },
  { key: 'done', label: 'เสร็จแล้ว' },
];

export default function FilterBar({ value, onChange }) {
  return (
    <View style={styles.row}>
      {FILTERS.map((f) => {
        const selected = f.key === value;
        return (
          <Pressable
            key={f.key}
            onPress={() => onChange(f.key)}
            style={[styles.chip, selected && styles.chipSelected]}
          >
            <Text style={[styles.label, selected && styles.labelSelected]}>
              {f.label}
            </Text>
          </Pressable>
        );
      })}
    </View>
  );
}

```

```
}

```

```
const styles = StyleSheet.create({
  row: { flexDirection: 'row', gap: 8, marginBottom: 12 },
  chip: { paddingHorizontal: 14, paddingVertical: 7, borderRadius: 999,
    borderWidth: 1, borderColor: COLORS.border,
    backgroundColor: COLORS.card },
  chipSelected: { borderColor: COLORS.cyan,
    backgroundColor: 'rgba(97, 218, 251, 0.14)' },
  label: { color: COLORS.textDim, fontSize: 14 },
  labelSelected: { color: COLORS.cyan, fontWeight: '700' },
});

```

8.3 เชื่อมเข้ากับหน้าจอหลัก

เพิ่มใน `TodoScreen.js` เพียงบรรทัดเดียวก็ได้ตัวกรองที่จำค่าไว้ข้ามการปิดแอป

```
import { useMemo } from 'react';
import { usePersistentState } from '../hooks/usePersistentState';
import FilterBar from '../components/FilterBar';

// ...ภายในคอมโพเนนต์
const [filter, setFilter] = usePersistentState(KEYS.FILTER, 'all');

// คำนวณรายการที่จะแสดงจาก todos และ filter
const visible = useMemo(() => {
  if (filter === 'active') return todos.filter((t) => !t.done);
  if (filter === 'done') return todos.filter((t) => t.done);
  return todos;
}, [todos, filter]);

```

แล้วเปลี่ยน `FlatList` ให้ใช้ `visible` แทน `todos` พร้อมวาง `FilterBar` ไว้เหนือรายการ

```
<FilterBar value={filter} onChange={setFilter} />

<FlatList
  data={visible}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <TodoItem item={item} onToggle={toggleTodo} onDelete={deleteTodo} />
  )}
  ListEmptyComponent={
    <Text style={styles.empty}>
      {filter === 'all' ? 'ลองเพิ่มรายการแรกดูสิ' : 'ไม่มีรายการในหมวดนี้'}
    </Text>
  }

```

```
keyboardShouldPersistTaps="handled"
/>
```

อย่าลืมเพิ่มสไตล์ `empty` ที่เพิ่งถูกอ้างถึง

```
empty: { color: COLORS.textDim, textAlign: 'center',
  marginTop: 40, fontSize: 15 },
```

ผลที่ควรเห็น

เลือกตัวกรองเป็น “ยังไม่เสร็จ” แล้วปิดแอปเปิดใหม่ แอปต้องกลับมาที่ตัวกรองเดิม ไม่ใช่ “ทั้งหมด” สังเกตว่าเราเพิ่มความสามารถนี้ด้วยโค้ดเพียงบรรทัดเดียว เพราะตรรกะการโหลดและบันทึกถูกซ่อนไว้ใน hook หมดแล้ว

useMemo มีไว้ทำอะไร

`useMemo` จะคำนวณ `visible` ใหม่ก็ต่อเมื่อ `todos` หรือ `filter` เปลี่ยนเท่านั้น ถ้าไม่ใช่ การกรองจะทำงานใหม่ทุกครั้งที่คอมโพเนนต์ render แม้จะเป็นการ render จากสาเหตุอื่นที่ไม่เกี่ยวข้องกัน ในรายการไม่กี่สับรายการอาจไม่รู้สึกรบกวน แต่เป็นนิสัยที่ควรติดไว้ตั้งแต่ต้น

9 ขั้นที่ 7 – ล้างข้อมูลอย่างปลอดภัย

ขั้นนี้จะได้ใช้คำสั่งชุดที่ทำงานกับหลายคีย์พร้อมกัน เพิ่มสองฟังก์ชันนี้เข้าไปใน `utils/storage.js`

```
// ล้างเฉพาะคีย์ของแอปนี้
// ไม่ใช่ AsyncStorage.clear() ซึ่งลบของไลบรารีอื่นไปด้วย
async clearAppData() {
  try {
    const keys = await AsyncStorage.getAllKeys();
    const mine = keys.filter((k) => k.startsWith(PREFIX));
    if (mine.length > 0) await AsyncStorage.multiRemove(mine);
    return mine.length;
  } catch (e) {
    console.warn('[storage.clearAppData] ล้มเหลว', e);
    return 0;
  }
},

// ใช้ตอนหาสาเหตุบั๊ก ดูว่ามีคีย์อะไรค้างอยู่ในเครื่องบ้าง
async debugDump() {
  const keys = await AsyncStorage.getAllKeys();
  return await AsyncStorage.multiGet(keys);
},
```


จากนั้นเพิ่มแถบปุ่มด้านล่างใน `TodoScreen.js`

```
import { Alert } from 'react-native';
import { BOTTOM_INSET } from '../constants/layout';

const clearCompleted = () => {
  const count = todos.filter((t) => t.done).length;
  if (count === 0) return;

  Alert.alert('ลบงานที่เสร็จแล้ว', `จะลบทั้งหมด ${count} รายการ`, [
    { text: 'ยกเลิก', style: 'cancel' },
    { text: 'ลบ', style: 'destructive',
      onPress: () => setTodos((prev) => prev.filter((t) => !t.done)) },
  ]);
};

const clearEverything = () => {
  Alert.alert('ล้างข้อมูลทั้งหมด', 'ข้อมูลในเครื่องจะถูกลบและกู้คืนไม่ได้', [
    { text: 'ยกเลิก', style: 'cancel' },
    { text: 'ล้าง', style: 'destructive',
      onPress: async () => {
        const n = await storage.clearAppData();
        setTodos([]);
        Alert.alert('เสร็จแล้ว', `ลบไป ${n} คีย์`);
      } },
  ]);
};

const showStoredKeys = async () => {
  const pairs = await storage.debugDump();
  const body = pairs.length
    ? pairs.map(([k, v]) => `${k}\n  ${String(v).slice(0, 60)}\n`).join('\n\n')
    : 'ยังไม่มีข้อมูลในเครื่อง';
  Alert.alert('ข้อมูลใน AsyncStorage', body);
};
```

แล้ววางแถบปุ่มไว้ล่างสุดของหน้าจอ

```
<View style={styles.footer}>
  <Pressable onPress={clearCompleted}>
    <Text style={[styles.linkText, { color: COLORS.amber }]}>ลบที่เสร็จแล้ว</Text>
  </Pressable>
  <Pressable onPress={showStoredKeys}>
    <Text style={[styles.linkText, { color: COLORS.violet }]}>ดูข้อมูลที่เก็บ</Text>
  </Pressable>
  <Pressable onPress={clearEverything}>
    <Text style={[styles.linkText, { color: COLORS.red }]}>ล้างทั้งหมด</Text>
  </Pressable>
</View>
```

```

footer: { flexDirection: 'row', justifyContent: 'space-around',
          borderTopWidth: 1, borderTopColor: COLORS.border,
          paddingTop: 12, paddingBottom: BOTTOM_INSET,
          paddingHorizontal: 12 },
linkText: { fontSize: 14, fontWeight: '600' },

```

ผลที่ควรเห็น

กดปุ่ม “ดูข้อมูลที่เก็บ” จะเห็นกล่องข้อความแสดงคีย์ `todo:items` และ `todo:filter` พร้อมค่าที่เก็บอยู่จริงในรูปแบบ JSON นี่คือการมีภาษาเหตุบังเอิญที่ดีที่สุดสำหรับงานลักษณะนี้ เพราะเห็นทันทีว่าสิ่งที่เขียนลงไปกับสิ่งที่คิดว่าเขียนตรงกันหรือไม่

ห้ามใช้ `AsyncStorage.clear` ในโค้ดจริง

`AsyncStorage.clear()` ลบทุกอย่างในพื้นที่เก็บข้อมูลของแอป รวมถึงข้อมูลที่ไลบรารีอื่นเก็บไว้ด้วย เช่น สถานะการนำทางที่ค้างไว้ หรือคิวข้อมูลของไลบรารีวิเคราะห์การใช้งาน การกรองด้วยคำแนะนำแล้วใช้ `multiRemove` จึงเป็นวิธีที่ถูกต้อง และเป็นเหตุผลที่เราตั้งคำแนะนำ `todo:` ไว้ตั้งแต่ขั้นที่ 1

9.1 จัดหน้าจอให้เรียบง่ายและตรวจ StyleSheet ฉบับสมบูรณ์

ถึงตรงนี้ `TodoScreen.js` ถูกแก้มาหลายรอบ จนสไตล์บางตัวถูกย้ายออกไปอยู่คอมโพเนนต์ลูกและบางตัวเพิ่งเพิ่มเข้ามา ให้เทียบกับฉบับสมบูรณ์ต่อไปนี้เพื่อความมั่นใจว่าตรงกับโครงการตัวอย่างทุกประการ

ปรับโครงของ `return` หลักให้แยกเป็นสามส่วนคือ ส่วนหัว ส่วนเนื้อ และแถบปุ่มล่าง พร้อมห่อด้วย `KeyboardAvoidingView` เพื่อไม่ให้แป้นพิมพ์บนระบบ iOS บังช่องกรอก

```

import { KeyboardAvoidingView, Platform } from 'react-native';

// ...ภายในคอมโพเนนต์ เพิ่มตัวนับงานที่เหลือ
const remaining = todos.filter((t) => !t.done).length;

return (
  <KeyboardAvoidingView
    style={styles.container}
    behavior={Platform.OS === 'ios' ? 'padding' : undefined}
  >
    <View style={styles.header}>
      <Text style={styles.title}>สิ่งที่ต้องทำ</Text>
      <Text style={styles.subtitle}>
        {todos.length === 0
          ? 'ยังไม่มีรายการ'
          : `เหลืออีก ${remaining} จาก ${todos.length} รายการ`}
      </Text>
    </View>

    <View style={styles.body}>

```

```

    <TodoInput onAdd={addTodo} />
    <FilterBar value={filter} onChange={setFilter} />
    <FlatList ... />
  </View>

  <View style={styles.footer}>
    ...ปุ่มสามปุ่มจากหัวข้อที่แล้ว
  </View>
</KeyboardAvoidingView>
);

```

สังเกตว่า container ไม่มี paddingTop และ paddingHorizontal อีกต่อไป เพราะย้ายไปอยู่ที่ header และ body แทน การแยกแบบนี้ทำให้เส้นคั่นของแถบปุ่มล่างลากได้เต็มความกว้างจอ ไม่ถูกระยะขอบตันเข้ามา

StyleSheet ฉบับสมบูรณ์ของ screens/ToDoScreen.js มีทั้งหมด 11 ตัว ดังนี้

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: COLORS.bg },
  center: { alignItems: 'center', justifyContent: 'center', gap: 12 },
  loadingText: { color: COLORS.textDim, fontSize: 14 },

  header: { paddingTop: TOP_INSET, paddingHorizontal: 20,
    paddingBottom: 16 },
  title: { color: COLORS.text, fontSize: 28, fontWeight: '800' },
  subtitle: { color: COLORS.textDim, fontSize: 14, marginTop: 4 },

  body: { flex: 1, paddingHorizontal: 20 },
  empty: { color: COLORS.textDim, textAlign: 'center',
    marginTop: 40, fontSize: 15 },

  footer: { flexDirection: 'row', justifyContent: 'space-around',
    borderTopWidth: 1, borderTopColor: COLORS.border,
    paddingTop: 12, paddingBottom: BOTTOM_INSET,
    paddingHorizontal: 12 },
  linkButton: { paddingVertical: 4, paddingHorizontal: 8 },
  linkText: { fontSize: 14, fontWeight: '600' },
});

```

สไตล์ที่ต้องไม่เหลืออยู่แล้ว

ถ้ายังมี row, input, addButton, addText, item, itemText หรือ itemDone ค้างอยู่ใน ToDoScreen.js แปลว่าลืมลบตอนแยกคอมโพเนนต์ในขั้นที่ 5 สไตล์เหล่านั้นย้ายไปอยู่ใน ToDoInput.js และ TodoItem.js แล้ว การปล่อยทิ้งไว้ไม่ทำให้แอปพัง แต่จะทำให้คนอ่านโค้ดต่อเข้าใจผิดว่ายังถูกใช้งานอยู่

10 รายการทดสอบก่อนส่งงาน

ให้ทดสอบทุกข้อและทำเครื่องหมายเมื่อผ่าน

1. เพิ่มรายการใหม่ได้ และช่องกรอกถูกล้างหลังเพิ่ม
2. กดปุ่มเพิ่มขณะช่องว่างแล้วไม่มีรายการว่างถูกสร้าง
3. แตะรายการแล้วสถานะเปลี่ยนเป็นขีดฆ่า แตะซ้ำแล้วกลับเป็นปกติ
4. กดปุ่มลบแล้วรายการนั้นหายไป และไม่มีรายการอื่นถูกลบตามไปด้วย
5. ปิดแอปจนสุดแล้วเปิดใหม่ รายการและสถานะติ๊กยังอยู่ครบ
6. เลือกตัวกรองแล้วปิดแอปเปิดใหม่ ตัวกรองยังเป็นค่าเดิม
7. เปิดแอปครั้งแรกหลังล้างข้อมูล ไม่มี error และแสดงข้อความว่ายังไม่มีรายการ
8. ปุ่ม “ลบที่เสร็จแล้ว” ลบเฉพาะรายการที่ติ๊กแล้วเท่านั้น
9. ปุ่ม “ล้างทั้งหมด” มีกล่องยืนยันก่อนลบเสมอ
10. ไม่มีการเรียก `AsyncStorage.clear()` ที่ใดในโค้ด
11. ไม่มีข้อความเตือนเรื่อง key ซ้ำจาก `FlatList` ใน console
12. เพิ่มรายการเร็ว ๆ 10 รายการติดกันแล้วยังทำงานถูกต้อง

11 โจทย์ต่อยอด

ระดับที่ 1 – แก้ไขข้อความของรายการ

ทำให้แต่ละค่าที่รายการแล้วเปลี่ยนเป็นโหมดแก้ไขข้อความได้ เมื่อแก้ไขเสร็จต้องบันทึกลงเครื่องเช่นเดียวกับการกระทำอื่น

ระดับที่ 2 – เรียงลำดับและกำหนดวันครบกำหนด

เพิ่มฟิลด์ `dueDate` ให้แต่ละรายการ พร้อมตัวเลือกเรียงลำดับตามวันที่สร้างหรือวันครบกำหนด และให้แอปจำตัวเลือกการเรียงลำดับไว้ด้วย

ข้อควรระวัง: การเพิ่มฟิลด์ใหม่ทำให้ข้อมูลที่บันทึกไว้ก่อนหน้านี้ไม่มีฟิลด์นั้น ต้องเขียนโค้ดรองรับกรณีที่ `dueDate` เป็น `undefined` ไม่ให้แอปพัง

ระดับที่ 3 – แยกเป็นหลายรายการและบันทึกร่าง

ขยายให้ผู้สร้างได้หลายรายการ (เช่น “งานบ้าน” “การเรียน”) โดยแต่ละรายการเก็บด้วยคีย์ของตัวเอง เช่น `todo:list:home` และมีคีย์ `todo:lists` เก็บรายชื่อทั้งหมด พร้อมทำให้ปุ่มล้างทั้งหมดยังทำงานถูกต้อง

เพิ่มเติม: ขณะพิมพ์งานใหม่ ให้บันทึกร่างอัตโนมัติแบบหน่วงเวลา 1 วินาที และเมื่อเปิดแอปใหม่ให้ถามว่าจะกู้ร่างที่ค้างไว้หรือไม่

ระดับที่ 4 – รองรับข้อมูลรูปแบบเก่า (สำหรับผู้สนใจ)

เมื่อแอปออกเวอร์ชันใหม่ที่เปลี่ยนโครงสร้างข้อมูล ผู้ใช้เดิมจะยังมีข้อมูลรูปแบบเก่าค้างอยู่ในเครื่อง โค้ดใหม่อ่านขึ้นมาแล้วอาจพังทันที

ลองจำลองสถานการณ์นี้ดู สมมติว่าเวอร์ชันแรกเก็บรายการเป็นอาร์เรย์ของสตริง ['ชื่อของ', 'ส่งงาน'] แต่เวอร์ชันปัจจุบันเก็บเป็นอาร์เรย์ของอ็อบเจกต์ จงเขียนฟังก์ชันที่ตรวจสอบข้อมูลรูปแบบเก่าแล้วแปลงให้ถูกต้อง โดยต้องทำงานถูกต้องแม้ผู้ใช้เปิดแอปซ้ำหลายครั้ง
แนวทางและตัวอย่างโค้ดอยู่ในเอกสารอ่านนอกเวลา หัวข้อ “การอัปเดตโครงสร้างข้อมูล”

12 ภาคผนวก – โครงการตัวอย่าง

โครงการใน `TodoStorageDemo.zip` ตรงกับโค้ดที่พิมพ์ในเอกสารนี้ทุกไฟล์ ใช้เทียบเมื่อทำแล้วติดขัด หรือใช้ตรวจงานของตัวเองหลังทำครบทุกขั้น

12.1 วิธีเปิดโครงการตัวอย่าง

```
unzip TodoStorageDemo.zip
cd TodoStorageDemo
npm install
npx expo install --fix      # ปรับทุกแพ็คเกจให้ตรงกับ SDK ที่ติดตั้ง
npx expo start
```

ต้องรัน `expo install --fix`

ไฟล์ `package.json` ในโครงการตัวอย่างระบุเวอร์ชันไว้ตามที่ทดสอบในขณะจัดทำเอกสาร คำสั่ง `npx expo install --fix` จะปรับทุกแพ็คเกจให้ตรงกับ Expo SDK ที่ติดตั้งอยู่จริงในเครื่อง ควรรันหนึ่งครั้งเสมอหลัง `npm install`

บทปฏิบัติการนี้ใช้ประกอบเอกสารอ่านนอกเวลา “การเก็บข้อมูลในเครื่องบน React Native + Expo”

วิชา 01418342 การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่ ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีดิจิทัล
คณะศิลปศาสตร์และวิทยาศาสตร์ มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน